

5. Generate a 32-octet random challenge `client_challenge := Crypto_DRBG(32 * 8)`
6. Invoke the `SignVIDVerificationRequest` command of the Operational Credential Cluster against the candidate using the `FabricIndex` of the entry in `Fabrics` whose `VendorID` is being verified, and the `client_challenge` generated in a previous step. This yields `SignVIDVerificationResponse(fabric_index, fabric_binding_version, signature)`.
7. Obtain the `attestation_challenge` from `CASE session`, `resumed CASE session` or `PASE session`, depending on the secure session context of the session over which the `SignVIDVerificationRequest` command was sent.
8. Locally generate the `vendor_fabric_binding_message`, following the `fabric_binding_version` obtained in the `SignVIDVerificationResponse`.
 - For a `fabric_binding_version` version of 0x01, generate `vendor_fabric_binding_message := fabric_binding_version (0x01) || root_public_key || fabric_id || vendor_id`
 - `root_public_key` is the entire octet string of the `RootPublicKey` field of the entry in `Fabrics` whose `VendorID` is being verified.
 - `fabric_id` is the 64-bit `FabricID` field of the entry in `Fabrics` whose `VendorID` is being verified, encoded as a big-endian value without omitting leading zeroes.
 - `vendor_id` is the 16-bit `VendorID` field of the entry in `Fabrics` whose `VendorID` is being verified, encoded as a big-endian value without omitting leading zeroes.
9. Locally generate the `vendor_id_verification_tbs` message
 - `vendor_id_verification_tbs := fabric_binding_version || client_challenge || attestation_challenge || fabric_index || vendor_fabric_binding_message || <vid_verification_statement>`
 - `fabric_binding_version` is the value from the `SignVIDVerificationResponse`.
 - `client_challenge` is the 32-octet string previously randomly generated.
 - `attestation_challenge` is the attestation challenge value previously obtained from the secure session context.
 - `fabric_index` is the 1-octet value from the `SignVIDVerificationResponse`
 - `vendor_fabric_binding_message` is the octet string generated in a previous step.
 - `vid_verification_statement` is the 85-octet (for cryptographic primitives mapping 1.0) value from the `VIDVerificationStatement` field of the entry in `Fabrics` whose `VendorID` is being verified, if it was present.
10. Given the subject public key associated with the fabric being verified, validate that `Crypto_Verify(noc_public_key, vendor_id_verification_tbs, signature)` succeeds, otherwise the procedure terminates as failed.
 - `noc_public_key` value is the subject key extracted from the `NOCValue` field of the entry in `NOCs` whose `FabricIndex` field matches the fabric being verified.
 - `vendor_id_verification_tbs` is the value generated in the previous step.
 - `signature` is the value from `SignVIDVerificationResponse` received previously.
11. Verify that the NOC chain is valid using `Crypto_VerifyChain()` against the entire chain reconstructed from both the `NOCs` and `TrustedRootCertificates` attribute entries whose `FabricIndex`